

ENGG1910 Demystifying AI

Lab3 Guidelines

1 Objectives

Text classification is the process of categorizing text into organized groups. In this lab, we will

1. go through the pre-process of the IMDB dataset,
2. test a long short-term memory (LSTM) model, which is a typical artificial recurrent neural network (RNN) architecture, to categorize the IMDB dataset,
3. test a mixed model [convolutional neural network (CNN) + LSTM] of this task
4. compare the output of these two models and tune for a better performance

2 Detailed Steps

2.1 Download IMDB dataset

Dataset from IMDB is used to make a recommendation system, but in this lab, we need to do a simple sentiment analysis first.

```
from keras.datasets import imdb
```

2.2 Read and explore dataset

1. Check the structure of the learning data

```
y_train[0:5]  
print(X_train[0])
```

2. Check the length of these data, i.e., number of items of different data

```
len(X_train[0]), len(X_train[1])
```

```
(74, 170)
```

← Their lengths are different.

3. Convert the training data into a standard format (Normalize the Data length)

```
len(X_train[0]), len(X_train[1])
```

```
(600, 600)
```

← Data lengths are same after normalization.

4. De-encode the training data to check the original data (task 1)

```
[8] decode_review(X_train[0])
```

```
'<START> this is one of those movies that you wish you hadn't seen before so you could see it again for the first time  
ing pleasure but are <UNK> <UNK> <UNK> by many this movie is by far the best film adaptation of his works william powe  
more the plot is intricate the story moves all too quickly because you want it to last enjoy'
```

2.3 LSTM model

2.3.1 Construct LSTM model

We will use *tensorflow/keras* to construct a LSTM model.

```
from keras.models import Sequential
from keras.layers import Dense,MaxPooling1D
from keras.layers import LSTM, Flatten, Dropout, Conv1D
from keras.layers.embeddings import Embedding
from keras.utils import sequence

# The code for LSTM Based Classification
embedding_vector_length = 32
model = Sequential()
model.add(Embedding(input_dim=top_words, output_dim=embedding_vector_length, input_length=max_review_length)) # Embedding layer
model.add(LSTM(100)) # LSTM Layer
model.add(Dense(units=1, activation='sigmoid')) # Dense Layer
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

2.3.2 Show the model structure

```
print(model.summary())
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 600, 32)	320000
lstm_2 (LSTM)	(None, 100)	53200
dense_2 (Dense)	(None, 1)	101
Total params: 373,301		
Trainable params: 373,301		
Non-trainable params: 0		

2.3.3 Start model tuning

```
history = model.fit(X_train, y_train, epochs=2, batch_size=32, verbose=1, validation_data=(X_cv, y_cv))
```

```
Epoch 1/2
547/547 [=====] - 370s 588ms/step - loss: 0.4734 - accuracy: 0.7733 - val_loss: 0.3213 - val_accuracy: 0.8688
Epoch 2/2
547/547 [=====] - 324s 592ms/step - loss: 0.2856 - accuracy: 0.8868 - val_loss: 0.3587 - val_accuracy: 0.8507
```

2.3.4 Optimize the model structure (task 2)

In this case, we can choose two series of hyperparameters to tune the LSTM network.

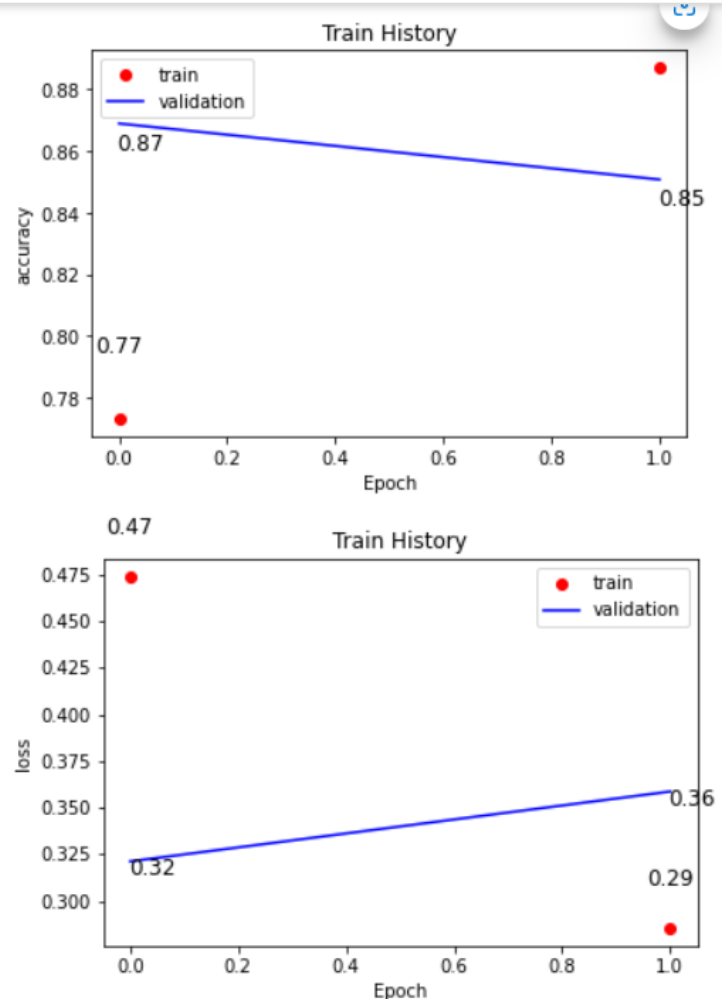
```
## You are free to choose the hyperparameters and the number of layers
## A. LSTM(100) epochs=2 batch_size=32
## B. LSTM(200) epochs=5 batch_size=64
```

In Task 2, students are required to train the model with these different hyperparameters and select an optimal one with a **lower** testing loss and **higher** testing accuracy.

2.3.5 Show the train and validation process (task 2)

```
show_train_history(history,'accuracy','val_accuracy')
```

```
show_train_history(history,'loss','val_loss')
```



This figure is important since it demonstrates the loss and accuracy over the train and validation set. The students are asked to upload their screenshot of this figure in task file.

2.3.6 Check the learned model in the test set (task 2)

```
# Check the learned model with test set
```

```
predict=model.predict(X_test)
```

```
predict_classes=np.argmax(predict,axis=1)
```

```

782/782 [=====] - 112s 144ms/step - loss: 0.3733 - accuracy: 0.8427
Testing loss      37.33219802379608
Testing accuracy  84.27199721336365

```

2.4 CNN+LSTM model

2.4.1 Construct CNN+LSTM model (task3)

create the model

embedding vector length = 128

cnn_model = Sequential()

Change the number of epochs and the batch size depending on the RAM Size

cnn_model.add(Embedding(input_dim=top_words, output_dim=embedding_vector_length, input_length=max review length)) # Embedding layer

cnn_model.add(Conv1D(filters=128, kernel_size=3, padding='same', activation='relu'))

cnn_model.add(MaxPooling1D(pool_size=1)) # Convolution-

1D Layer : You are free to choose the hyperparameters and the number of layers

cnn_model.add(Conv1D(filters=64, kernel_size=2, padding='same', activation='relu'))

cnn_model.add(MaxPooling1D(pool_size=1))

cnn_model.add(LSTM(50))

LSTM Layer : You are free to choose the hyperparameters and the number of layers

cnn_model.add(Dense(units=1, activation='sigmoid')) # Dense Layer

cnn_model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

print(cnn_model.summary())

We can see this network structure as

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(None, 600, 128)	1280000
conv1d_2 (Conv1D)	(None, 600, 128)	49280
max_pooling1d_2 (MaxPooling 1D)	(None, 600, 128)	0
lstm_4 (LSTM)	(None, 50)	35800
dense_4 (Dense)	(None, 1)	51
=====		
Total params: 1,365,131		
Trainable params: 1,365,131		
Non-trainable params: 0		

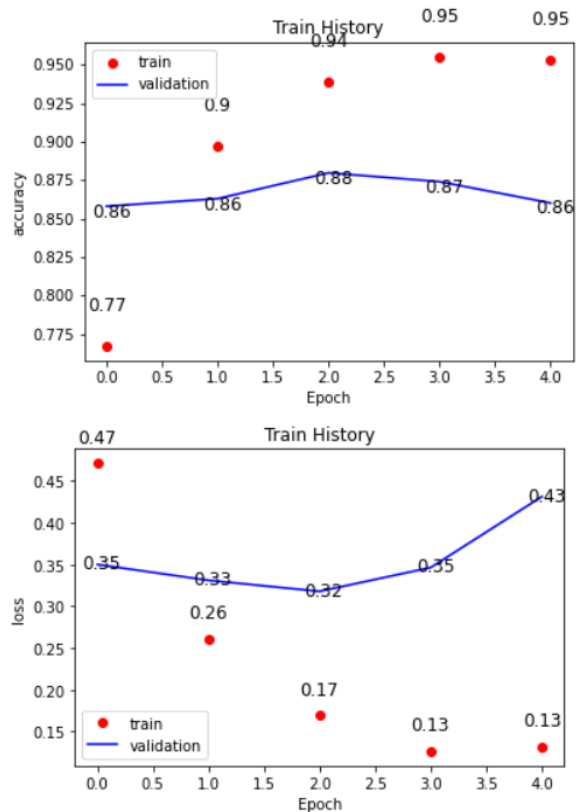
Students are required to record their CNN+LSTM model hyperparameters in task3. For more details, we kindly refer interested students to check the task file.

2.4.2 Show the train and validation process

Final evaluation of the CNN + LSTM model using the test data

show_train_history(history_c, 'accuracy', 'val_accuracy')

show_train_history(history_c, 'loss', 'val_loss')



2.4.3 Show the result

Final evaluation of the model using test dataset

```
cnn_scores = cnn_model.evaluate(X_test, y_test, verbose=1)
```

```
print('Testing loss \t', scores[0]*100)
```

```
print('Testing accuracy ', scores[1]*100)
```

```
782/782 [=====] - 95s 122ms/step - loss: 0.4981 - accuracy: 0.8391
```

```
Testing loss      37.33219802379608
```

```
Testing accuracy  84.27199721336365
```

3 Compartion (task4)

We have provided the defined function `show_values_on_bars()` such that students can store the previous data in the variable `results`.

```
results=[]
```

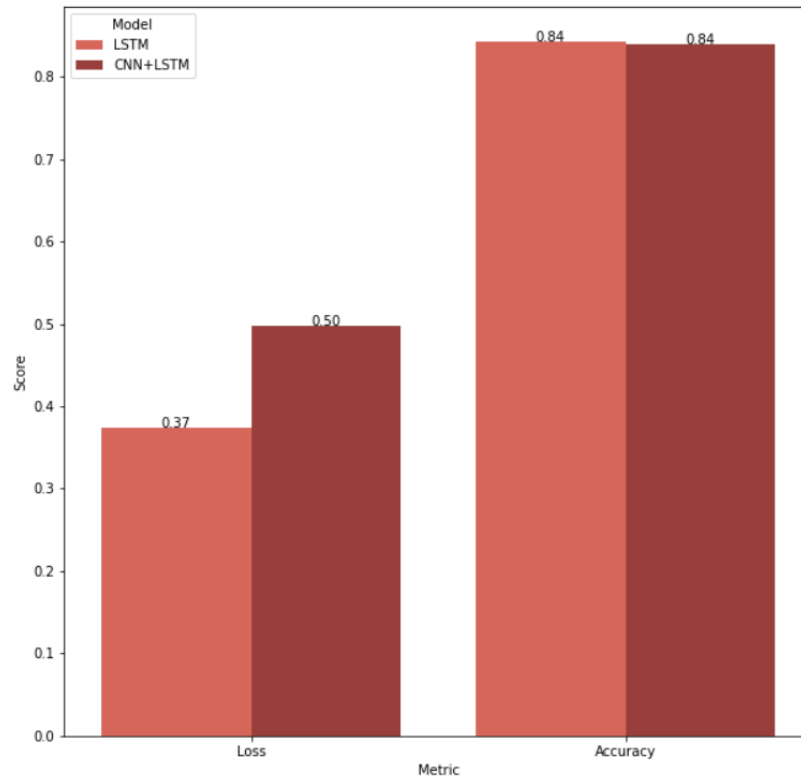
```
results.append([scores[0], 'Loss', 'LSTM'])
```

```
results.append([scores[1], 'Accuracy', 'LSTM'])
```

```
results.append([cnn_scores[0], 'Loss', 'CNN+LSTM'])
```

```
results.append([cnn_scores[1], 'Accuracy', 'CNN+LSTM'])
```

Then, we can generate a bar as below:



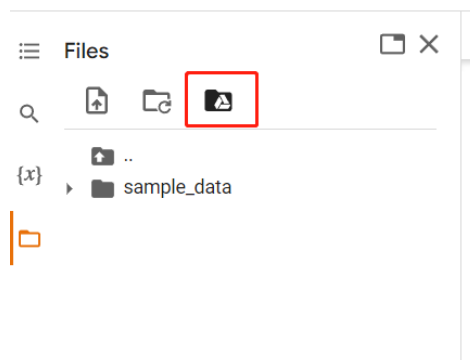
In task5, students are required to copy and paste their result of the LSTM and CNN+LSTM model.

4 Hints of the Tasks

Lab3 tasks file contains a detailed description of the lab task, students need to submit it on Blackboard as a single PDF file.

Task5 is an optional task, we recommend the students who have a basic idea about python, matplotlib and neural network tuning in advanced, to implement this task.

- Hint 1: Keep a regular backup of your trained model
- Hint 2: Link your colab notebook to Google drive



- Hint 3: Save your TensorFlow model in Google drive.

```
cd /content/drive/MyDrive/lab4 # As an example, to help you store the data
```

```
# Save the trained model
```

```
model.save('text_classification.h5')
```

```
# Load the trained model
```

```
# pre_trained_model = tf.keras.models.load_model('text_classification.h5')
```

- Hint 4: The loss of validation shows the fitting situation of the training process, if it gradually increases over epochs, that means the model is overfitting.
- Hint 5: Limited by the Colab power, students can do these actions to speed up the training process
 - decrease epoch
 - decrease batch_size
 - lower the LSTM number, i.e., from LSTM(200) to LSTM(100)
 - lower the embedding_vector_length